

CD Pipeline Approach Document

1. Executive Summary

This document covers the Continuous Deployment (CD) approach using Argo CD on GCP. CD begins only after Jenkins CI completes successfully and a verified container image is published to Artifact Registry.

The approach is GitOps-first:

- Git is the single source of truth for all deployed state.
- All deployments are performed by Argo CD — no manual kubectl commands in any environment.
- Dev auto-deploys on every successful CI run. UAT requires a manual trigger. Prod requires an explicit approval gate.
- All environments use rolling deployments for zero downtime.
- Argo CD continuously reconciles live cluster state against Git — any drift is detected and corrected automatically.

2. Pre-Requisites

Items the infrastructure team must provide before Argo CD setup begins. Installation and configuration of Argo CD is handled by the DevOps team.

2.1 Argo CD Cluster

- A dedicated GKE cluster provisioned in the CI/CD control plane VPC for Argo CD to run on.
- Cluster must have workload identity enabled and sufficient node capacity (min 2 vCPU / 4 GB RAM per node, min 2 nodes).
- Namespace 'argocd' pre-created with appropriate resource quotas.
- Argo CD installation and configuration will be handled by the DevOps team.
- An Ingress controller (NGINX / GKE native / Traefik) must be installed on the Argo CD GKE cluster prior to setup. This is required to expose the Argo CD UI and API server endpoint within the private network for DevOps team access and Jenkins CD trigger connectivity.

2.2 Target Cluster Access

Argo CD must be able to connect to the Dev, UAT, and Prod GKE clusters. The infrastructure team must provide:

- Dev, UAT, and Prod GKE clusters provisioned and reachable from the Argo CD cluster over private Google backbone (no public endpoint required).
- A service account created in each target cluster (dev, uat, prod) with namespace-scoped RBAC — permission to apply, patch, and delete resources in the target namespace only.
- Kubeconfig or cluster API endpoint details shared with the DevOps team for cluster registration in Argo CD.

2.3 Networking

The following network paths must be open before setup:

From	To	Port / Protocol	Purpose
Argo CD cluster (control VPC)	Dev GKE cluster API server	443 / HTTPS (private)	Argo CD syncs Dev deployments
Argo CD cluster (control VPC)	UAT GKE cluster API server	443 / HTTPS (private)	Argo CD syncs UAT deployments
Argo CD cluster (control VPC)	Prod GKE cluster API server	443 / HTTPS (private)	Argo CD syncs Prod deployments
Jenkins VM (control VPC)	Argo CD server	443 / HTTPS (private)	Jenkins fires CD trigger
Argo CD cluster	Artifact Registry endpoint	443 / HTTPS	Pull image metadata / OCI ref
Argo CD cluster	Git repository (SCM)	443 / HTTPS	Watch manifests and Helm charts

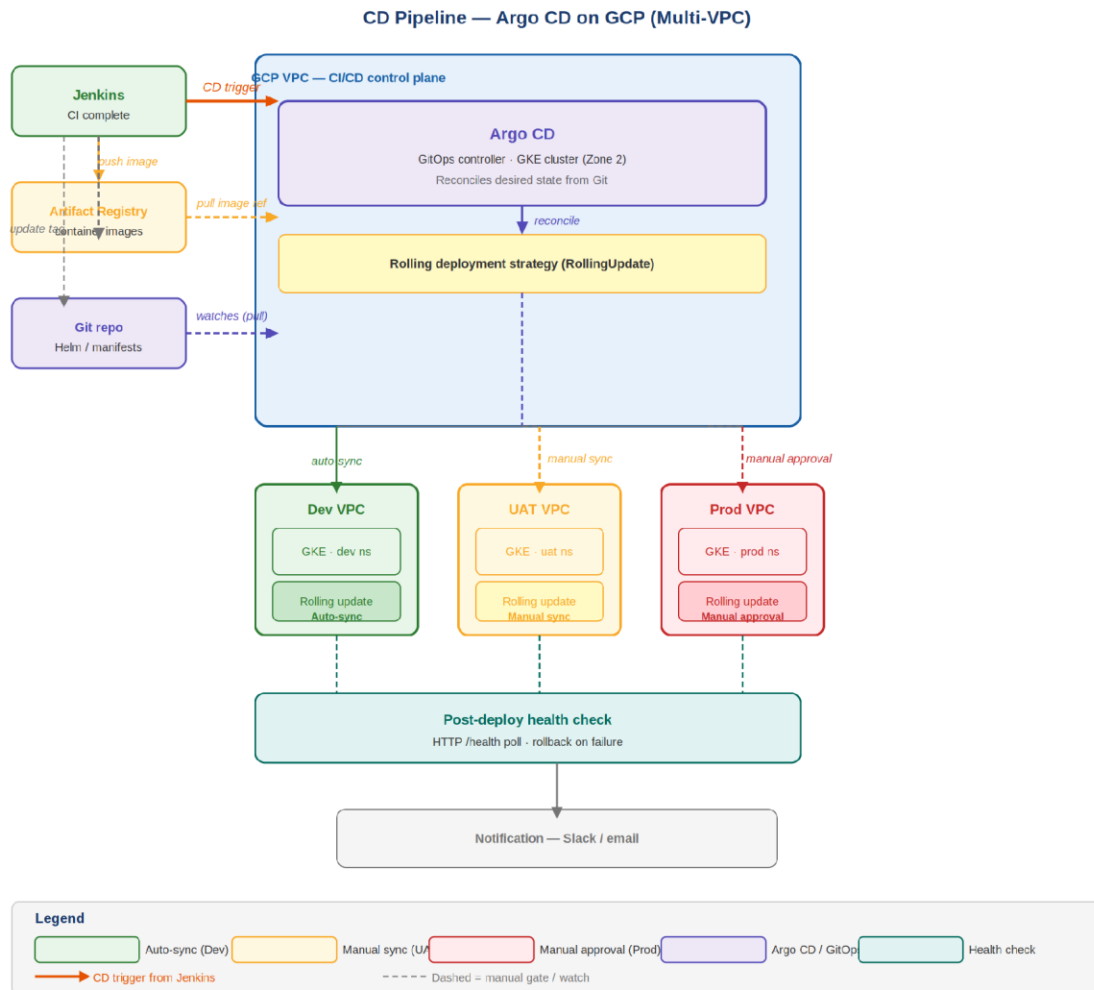
2.4 Container Registry

- Container Registry provisioned.
- Pull access granted to all three target cluster node service accounts so pods can pull images at runtime.

2.5 Git / SCM

- A dedicated GitOps repository created (separate from application source code).
- DevOps team granted write access to the GitOps repository.
- Jenkins CI service account granted read + tag-push access to the GitOps repository.

3. CD Architecture



4. GitOps Model

4.1 Source of Truth

- All desired state is declared in the GitOps repository as Helm charts and Kubernetes manifests.
- No environment is changed by any mechanism other than Argo CD reading from Git.
- Rollback = revert a Git commit. Argo CD applies the previous state automatically.

Key rule	No direct kubectl apply commands in any environment. Every change is a Git commit.
----------	--

4.2 Repository Structure

Path	Contents	Updated by
charts/<service>/	Helm chart templates	DevOps team
charts/<service>/values-dev.yaml	Dev resource sizing, feature flags, ingress	DevOps team
charts/<service>/values-uat.yaml	UAT values	DevOps team
charts/<service>/values-prod.yaml	Prod values (HPA, PDB, limits)	DevOps team

5. CD Trigger — Jenkins Handoff

1. CI completes all 7 stages. Verified image pushed to Artifact Registry with version tag.
2. Jenkins commits updated image tag to values-dev.yaml in the GitOps repo.
3. Jenkins fires: argocd app sync <service>-dev triggering immediate reconciliation.
4. Argo CD detects the new desired state and begins rolling deployment to Dev.
5. Deployment outcome (pass / fail / rollback) notified to Slack and logged.

5.1 Branch-to-Environment Mapping

Branch event	CD action	Environment	Sync mode
Merge to Dev branch	Jenkins triggers argocd sync	Dev VPC	Auto — immediate
Dev → UAT (merge)	Manual sync trigger in Argo CD	UAT VPC	Manual — team lead triggers
UAT → Main (tag)	Manual approval gate then sync	Prod VPC	Manual — explicit approval required

6. Environment Promotion Strategy

6.1 Dev — Auto-sync

- Argo CD auto-sync enabled. Every successful CI merge to Dev triggers an immediate deployment.
- Self-heal on: any drift from Git is automatically corrected.
- No human approval needed.

6.2 UAT — Manual Sync

- After Dev validation, a team lead updates values-uat.yaml with the validated image tag via pull request.
- Team lead manually triggers Argo CD sync for the UAT application via UI or CLI.
- Prevents any unvalidated build from reaching UAT automatically.

6.3 Prod — Manual Approval Gate

- A semantic version tag is cut from Main after UAT sign-off.
- PR raised to update values-prod.yaml. Requires approval from Dev Lead and Ops Lead.
- After merge, an authorised team member triggers Prod sync in Argo CD UI.
- Auto-sync is explicitly disabled for Prod. No automated deployment path to production exists.

Production safety

Auto-sync is disabled on the Prod Argo CD application. Only a named approver can initiate a production deployment. All actions are captured in the Argo CD audit log.

7. Rolling Deployment

All environments use Kubernetes RollingUpdate (maxSurge=1, maxUnavailable=0) ensuring zero downtime. New pods must pass readiness probes before old pods are terminated.

7.1 Deployment Verification & Rollback

- Argo CD monitors rollout status. Failure to complete within progressDeadline marks the sync as failed.
- Post-sync health check polls /health endpoint. Non-200 triggers automatic rollback via helm rollback.
- Notification dispatched to Slack and email on every deployment outcome.

8. Argo CD Configuration

8.1 Application Definitions

Application	Source path	Target cluster	Sync policy
<service>-dev	charts/<service> + apps/dev/	Dev GKE	Auto + self-heal + prune
<service>-uat	charts/<service> + apps/uat/	UAT GKE	Manual only
<service>-prod	charts/<service> + apps/prod/	Prod GKE	Manual + approval gate

8.2 Sync Policy Summary

Setting	Dev	UAT	Prod
Auto-sync	Enabled	Disabled	Disabled
Self-heal	Enabled	Disabled	Disabled
Prune	Enabled	Disabled	Disabled
Retry limit	3	2	1

8.3 Notifications

Event	Channel	Recipients
Sync succeeded	#deploy-<env>	DevOps team + service owner
Sync failed	#deploy-<env>	DevOps team + on-call
Health check failed	#deploy-alerts	On-call + Ops Lead
Rollback triggered	#deploy-alerts	On-call + Ops Lead + Dev Lead
Prod approval needed	#deploy-prod	Dev Lead + Ops Lead

9. Application Readiness Requirements

Each service must meet these requirements before it is onboarded to the CD pipeline.

- Exposes /health or /healthz returning HTTP 200 within 5 seconds.
- Readiness and liveness probes defined in the Helm chart values.

- CPU and memory requests and limits set per environment in Helm values files.
- Service is stateless OR has a documented persistent storage strategy (PVC / managed DB).
- No hardcoded secrets. All credentials injected at runtime via External Secrets Operator from Secret Manager.

End of Document

COE – Infrastructure & DevOps Division | Confidential